

Phase 1 Project Proposal

Project Title

Urban Traffic and Air Pollution: An End-to-End Distributed Data Engineering Pipeline

1. Domain & Source Identification

1.1 Project Concept

This project focuses on **urban transportation, air quality, and environmental data engineering**.

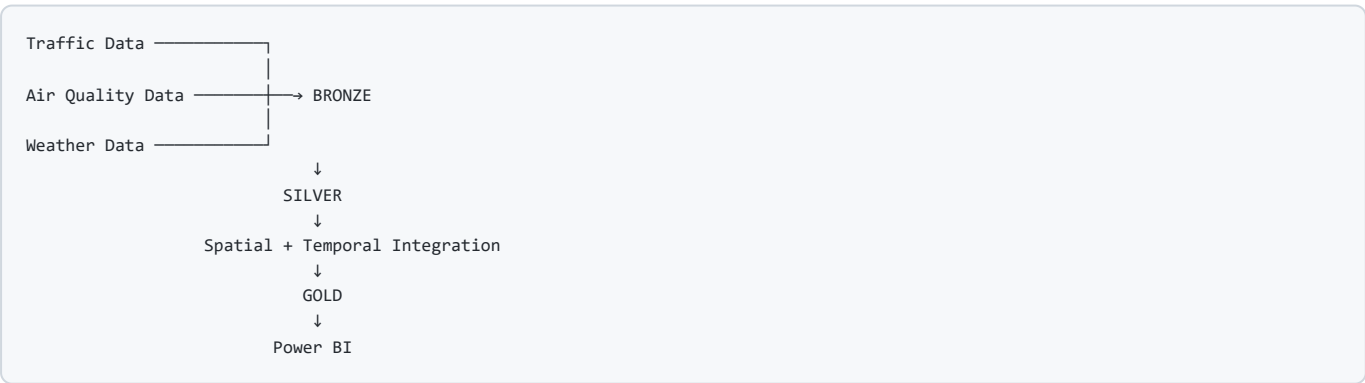
The objective is to design and implement an end-to-end automated data pipeline using **Apache Spark** that integrates traffic, air-quality, and weather data for London.

The pipeline will investigate the following analytical question:

How are traffic conditions spatially and temporally associated with nearby PM2.5 concentrations, and how do weather conditions correspond with this relationship?

The project will not claim that traffic alone causes changes in PM2.5. Instead, it will identify and visualize **observed spatial and temporal associations** while considering weather variables such as wind speed, wind direction, temperature, and precipitation.

The project will follow the **Medallion Architecture**:



The project is designed as a substantial Data Engineering system rather than only a visualization project. It will demonstrate:

- REST API ingestion
 - Full-load processing
 - Incremental ingestion
 - Distributed processing with Apache Spark
 - Data quality and cleansing
 - Schema standardization
 - Spatial processing
 - Temporal joins
 - Aggregation
 - Medallion Architecture
 - Performance optimization
 - Power BI analytics
 - GitHub-based version control
-

1.2 Data Sources

The project will use three primary public data sources.

A. Transport for London (TfL) Unified API — Traffic

The primary traffic source will be the **Transport for London (TfL) Unified API**.

Official API:

<https://api.tfl.gov.uk/>

TfL provides public open-data feeds through its Unified API. Developers can register for an application ID and key and access the API programmatically.

The API includes a **Road** resource. It provides:

- Roads managed by TfL
- Road identifiers and names
- Geographic bounds
- Road status
- Status severity
- Road disruptions
- Disruption location and coordinates
- Disruption timestamps
- Historical/date-range road status queries

The `Road/{ids}/Status` endpoint supports querying road status aggregated over a specified date range. The same API also provides current road status when no historical date range is supplied.

This allows the project to implement both:

Full Load

- Retrieve a historical baseline of London road-status information over a defined historical period.

Incremental Load

- Periodically retrieve newly available/current road-status and disruption information and append only new observations to the Bronze layer.

TfL states that its public data is freely released for developers and recommends the Unified API for live feeds.

B. OpenAQ — Air Quality

Air-quality data will be obtained from the **OpenAQ API v3**.

Official API documentation:

<https://docs.openaq.org/>

The project will primarily use **PM2.5** measurements from monitoring locations in London.

Relevant fields include:

- Sensor ID
- Location ID
- Latitude
- Longitude
- Measurement value
- Unit
- Parameter
- Measurement timestamp

OpenAQ provides measurement resources as well as hourly and daily aggregation resources. Its API supports geographic and date-range filtering, and large results can be retrieved through pagination.

The project will use historical PM2.5 observations for the full load and newly available observations for incremental ingestion.

C. Open-Meteo — Weather

Weather data will be obtained from the **Open-Meteo Historical Weather API**.

Official documentation:

<https://open-meteo.com/en/docs/historical-weather-api>

The project will use hourly weather variables such as:

- Temperature
- Wind speed
- Wind direction
- Precipitation
- Relative humidity

Historical weather data will provide the baseline for the selected London study period. New weather observations will be periodically retrieved for incremental processing.

1.3 Ingestion Pattern

The pipeline will support both **Full Load** and **Incremental Load**.

Full Load

The initial execution will retrieve a historical baseline.

The target historical period will initially be approximately **one year**, subject to the final availability and volume of the selected traffic and air-quality data.

The full-load process will:

1. Retrieve historical TfL road-status data.
2. Retrieve historical London PM2.5 measurements from OpenAQ.
3. Retrieve historical hourly weather data from Open-Meteo.
4. Store the raw responses in the Bronze layer.
5. Record an ingestion timestamp and source identifier for traceability.

Incremental Load

After the historical baseline has been loaded, subsequent executions will retrieve only newly available data.

The incremental process will maintain a **watermark**, such as the latest successfully processed timestamp.

Example:

```
Initial Full Load
2025-01-01 → 2025-12-31
↓
BRONZE

Incremental Run 1
2026-01-01 → latest
↓
BRONZE

Incremental Run 2
new records after previous watermark
↓
BRONZE
```

The pipeline will:

1. Read the latest processed timestamp.
2. Request data after that timestamp where supported.
3. Retrieve current/new traffic, air-quality, and weather observations.
4. Remove duplicate records.
5. Append valid new records to the Bronze layer.
6. Process only the affected data through Silver and Gold transformations.

This demonstrates a real incremental ingestion pattern rather than repeatedly reprocessing the complete historical dataset.

2. Data Samples & Volume

2.1 Sample Raw Data Files

The GitHub repository will contain raw sample files representing both the initial full load and a later incremental load.

Planned structure:

```
data/
├── full_load/
│   ├── traffic_full_sample.json
│   ├── air_quality_full_sample.json
│   └── weather_full_sample.json
└── incremental_load/
    ├── traffic_incremental_sample.json
    ├── air_quality_incremental_sample.json
    └── weather_incremental_sample.json
```

The samples will be obtained directly from the selected APIs and will preserve the source structure as much as practical.

The full-load samples will represent historical records, while the incremental samples will represent records retrieved after the full-load watermark.

2.2 Expected Data Volume

The project will use a sufficiently large dataset to demonstrate distributed processing while remaining manageable on Databricks Free Edition.

The initial target is approximately:

2–10 GB of raw data

across the three sources.

The exact volume will be finalized after source profiling because API response sizes depend on the selected historical period, number of road corridors, air-quality sensors, and weather coordinates.

Traffic Data

TfL road-status and disruption data will be collected for all 24 major TfL corridors (Red Routes).

The historical extraction will be divided into manageable date ranges and API requests to avoid oversized responses. Estimated volume: **500 MB – 2 GB** depending on historical depth.

Air-Quality Data

OpenAQ provides sensor-level measurements and supports pagination and date-range filtering. The project will focus on London PM2.5 sensors rather than downloading the global OpenAQ dataset.

The initial target is approximately:

1–5 GB

of raw air-quality data, depending on the selected historical period and sensors.

Weather Data

Open-Meteo hourly weather data will be collected for selected coordinates covering London.

The expected volume is approximately:

100 MB–1 GB

depending on the number of coordinates and historical period.

Incremental Volume

The expected incremental volume will initially be targeted at approximately:

50–300 MB per ingestion cycle

with the exact amount determined after the first live ingestion tests.

The project will prioritize realistic data volume and reproducible processing instead of artificially inflating the dataset size.

3. Security & Compliance

3.1 PII and Sensitive Data

The selected datasets are public transportation, environmental, and weather datasets.

The pipeline does not require personal information such as:

- Names
- Email addresses
- Phone numbers
- Financial information
- User passwords
- Personal account information

Traffic information represents road-level conditions and disruptions rather than identifying individual drivers.

Air-quality information represents measurements from environmental sensors.

Weather information represents environmental observations associated with geographic locations.

Therefore, the project does not expect meaningful Personally Identifiable Information (PII) in the analytical data.

3.2 Handling Strategy

Although PII is not expected, the pipeline will still apply data-governance controls.

The Silver layer will:

- Remove unnecessary source metadata.
- Retain only fields required for analysis.
- Validate geographic coordinates.
- Validate timestamps.
- Remove malformed records.
- Remove duplicate observations.
- Standardize measurement units.
- Keep API credentials outside the repository.

API keys and other credentials will **never be hard-coded into GitHub notebooks or source files**.

If unexpected PII is discovered in any raw source payload, the affected field will be removed or masked before the data is promoted to the Silver layer.

4. High-Level Medallion Data Modeling

4.1 Bronze Layer — Raw Data

The Bronze layer will contain minimally transformed data directly obtained from the external sources.

The primary Bronze datasets will be:

```
bronze_traffic  
bronze_air_quality  
bronze_weather
```

Bronze Traffic

Potential fields:

```
road_id  
display_name  
group  
status_severity  
status_severity_description  
bounds  
status_aggregation_start  
status_aggregation_end  
source  
ingestion_timestamp
```

For road disruptions, additional fields may include:

```
disruption_id
latitude
longitude
category
severity
status
start_datetime
end_datetime
last_modified_datetime
location
```

Bronze Air Quality

Potential fields:

```
sensor_id
location_id
timestamp
latitude
longitude
parameter
value
unit
source
ingestion_timestamp
```

Bronze Weather

Potential fields:

```
timestamp
latitude
longitude
temperature
wind_speed
wind_direction
precipitation
relative_humidity
source
ingestion_timestamp
```

The Bronze layer will preserve source-level information so that data lineage and debugging remain possible.

4.2 Silver Layer — Cleansed Data

The Silver layer will contain validated and standardized records.

Major transformations will include:

Schema Standardization

- Convert timestamps to Spark `TimestampType`.
- Convert numerical values to appropriate numeric types.
- Standardize column names.
- Standardize measurement units.
- Normalize latitude and longitude.
- Add source and ingestion metadata.

Data Quality

The pipeline will check for:

- Missing timestamps
- Invalid coordinates
- Duplicate records
- Invalid numeric values
- Missing sensor identifiers
- Corrupt API records
- Invalid measurement units

Invalid or unusable records will either be corrected where possible or excluded according to documented rules.

Time Standardization

Traffic, air-quality, and weather datasets may use different timestamp formats and time zones.

The Silver layer will convert timestamps to a common standard, preferably **UTC**, while retaining London local-time information where required for dashboard analysis.

OpenAQ explicitly provides both UTC and local timestamps and uses ISO-8601 datetime formats, making explicit timezone handling important when joining datasets.

The analytical time grain will initially be **hourly**.

4.3 Silver Traffic Model

The cleaned traffic data will be represented using a model similar to:

```
silver_traffic
-----
road_id
road_name
status_severity
status_description
latitude
longitude
status_start
status_end
timestamp
ingestion_timestamp
```

Road disruption records will be represented separately if required:

```
silver_traffic_disruptions
-----
disruption_id
road_id
latitude
longitude
category
severity
status
start_datetime
end_datetime
last_modified_datetime
```

This allows traffic conditions and traffic incidents to be analyzed independently and later integrated.

4.4 Silver Air-Quality Model

```
silver_air_quality
-----
sensor_id
location_id
timestamp
latitude
longitude
parameter
pm25_value
unit
ingestion_timestamp
```

Only PM2.5 measurements will be used for the primary project analysis.

4.5 Silver Weather Model

```
silver_weather
-----
timestamp
latitude
longitude
temperature
wind_speed
wind_direction
precipitation
relative_humidity
ingestion_timestamp
```

Weather records will be aligned to the hourly analytical time grain.

4.6 Spatial Processing

A major Spark challenge in this project will be the **spatial join** between London roads/traffic events and nearby air-quality sensors.

For example:

```
Traffic Road / Event
  |
  | distance
  |
Air Quality Sensor
```

The pipeline will identify traffic observations occurring within a predefined radius of an air-quality sensor.

An initial matching radius of approximately **1 kilometer** will be tested and adjusted based on data density and project requirements.

The resulting relationship will contain information such as:

```
sensor_id
road_id
distance_km
timestamp
traffic_status
PM2.5
```

This spatial relationship is one of the project's primary distributed-processing challenges.

4.7 Temporal Integration

Spatial matching alone is not sufficient.

A traffic event at 8:00 AM should primarily be compared with air-quality observations around the same period.

Therefore, the project will also perform a temporal join.

The integration will conceptually follow:

```
Traffic:
Road A
08:00
Heavy

      +

Air Quality:
Sensor S1
08:00
PM2.5 = 38

      +

Weather:
08:00
Wind = 7 km/h

      ↓

Integrated Observation
```

The final matching logic will use an hourly analytical grain and documented time-tolerance rules where exact timestamps do not match.

4.8 Gold Layer — Business-Ready Data

The Gold layer will contain consolidated datasets optimized for analytics and Power BI.

The primary table will be:

gold_traffic_air_quality

Potential fields:

```
date
hour
sensor_id
road_id
latitude
longitude
distance_km
traffic_status
traffic_severity
pm25
wind_speed
wind_direction
temperature
precipitation
```

Additional calculated fields may include:

```
traffic_category
pm25_category
hour_of_day
day_of_week
```

4.9 Gold Aggregations

gold_hourly_pollution

```
sensor_id
date
hour
avg_pm25
max_pm25
min_pm25
```

gold_traffic_pollution_summary

```
date
hour
traffic_category
avg_pm25
max_pm25
road_count
sensor_count
```

gold_location_summary

```
sensor_id
location
avg_pm25
heavy_traffic_percentage
avg_wind_speed
observation_count
```

These Gold datasets will prevent Power BI from repeatedly performing expensive transformations on raw or Silver data.

5. Business Intelligence & Dashboard

5.1 Dashboard Objective

The final dashboard will provide an interactive view of traffic conditions, PM2.5 concentrations, and weather conditions across London.

The dashboard will focus on **observed relationships and patterns**, not causal claims.

5.2 Business Questions

The dashboard will answer questions such as:

1. How does PM2.5 concentration vary across different traffic conditions?
 2. Which areas show both higher traffic severity and higher PM2.5 measurements?
 3. How do traffic and PM2.5 levels change throughout the day?
 4. How do wind speed and wind direction correspond with PM2.5 observations?
 5. Which locations and time periods have the highest observed PM2.5 levels?
 6. How frequently do high-pollution observations coincide with heavy traffic conditions?
-

5.3 Planned Visualizations

Visualization 1 — Traffic vs PM2.5

A scatter plot will compare:

X-axis → Traffic severity / traffic condition
Y-axis → PM2.5 concentration

Filters will allow users to select:

- Date
 - Hour
 - Sensor
 - Location
 - Weather conditions
-

Visualization 2 — London Pollution and Traffic Map

An interactive map will display air-quality sensors and nearby traffic conditions.

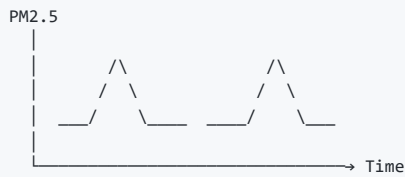
The map will help identify areas where:

- Traffic is high and PM2.5 is high
 - Traffic is high and PM2.5 is lower
 - Traffic is lower and PM2.5 is high
 - Traffic is lower and PM2.5 is lower
-

Visualization 3 — Time-Series Analysis

A time-series chart will show PM2.5 and traffic conditions over time.

Example:



The user will be able to filter by sensor, road, day, and traffic category.

5.4 Dashboard KPIs

The dashboard will include metrics such as:

- Average PM2.5
 - Maximum PM2.5
 - Average traffic severity
 - Percentage of observations associated with high traffic conditions
 - Number of air-quality sensors
 - Number of monitored roads
 - Number of spatially matched traffic-air-quality observations
 - Number of observations processed
-

6. Engineering Setup & FinOps

6.1 Technology Stack

Component	Technology
Distributed Processing	Apache Spark
Programming	Python / PySpark
Cloud Platform	Databricks Free Edition
Storage	Delta Lake / Parquet
Architecture	Medallion Architecture
Visualization	Microsoft Power BI
Version Control	GitHub
Traffic	Transport for London Unified API
Air Quality	OpenAQ API v3
Weather	Open-Meteo Historical Weather API

6.2 GitHub Repository

A GitHub repository will be created for the project.

Planned structure:

```
urban-traffic-air-pollution/
├── README.md
├── data/
│   ├── full_load/
│   │   ├── traffic_full_sample.json
│   │   ├── air_quality_full_sample.json
│   │   └── weather_full_sample.json
│   └── incremental_load/
│       ├── traffic_incremental_sample.json
│       ├── air_quality_incremental_sample.json
│       └── weather_incremental_sample.json
├── notebooks/
│   ├── 01_bronze_ingestion.py
│   ├── 02_silver_cleaning.py
│   ├── 03_spatial_temporal_join.py
│   └── 04_gold_aggregation.py
├── src/
│   ├── ingestion/
│   ├── transformations/
│   └── utilities/
├── dashboard/
└── docs/
    └── phase1_proposal.md
```

GitHub Repository: <https://github.com/Muhammad-Ibrahim-001/UrbanLake>

6.3 FinOps and Resource Management

The project will be developed using **Databricks Free Edition** or another approved educational cloud environment.

Because free-tier infrastructure has limited compute and storage resources, the team will use several resource-management strategies.

Controlled Historical Window

The initial full-load period will be limited to a practical historical window rather than downloading unnecessary data.

API Filtering

Only relevant:

- London geographic areas
- PM2.5 measurements
- required weather variables
- required traffic records

will be retrieved.

Development Sampling

Small samples will be used while developing and debugging transformations.

For example:

```
Small API sample
↓
Develop transformation
↓
Validate output
↓
Run larger dataset
```

Partitioning

Large Spark datasets will be partitioned using suitable fields such as:

```
year
month
date
```

This will reduce unnecessary data scanning.

Incremental Processing

After the initial full load, the pipeline will process only newly available records using a timestamp watermark.

Join Optimization

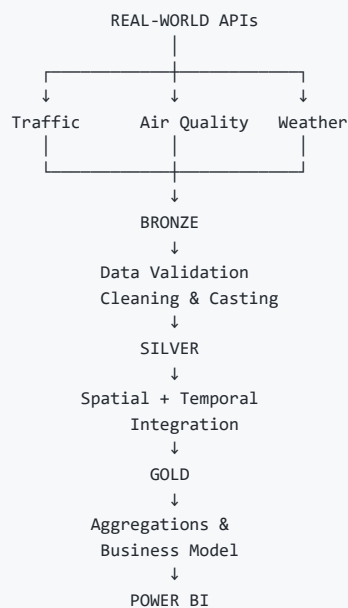
Spatial and temporal joins can be computationally expensive. The project will investigate Spark optimization techniques including:

- filtering before joins
- selecting only required columns
- appropriate partitioning
- repartitioning where necessary
- broadcasting small reference datasets where appropriate
- reducing unnecessary shuffles
- pre-aggregation before expensive joins

These techniques will allow the team to demonstrate not only that the pipeline works, but also how distributed processing performance can be improved.

7. Expected Outcome

The completed system will provide an automated end-to-end data pipeline:



The project will demonstrate how Apache Spark can be used to integrate heterogeneous, geographic, and time-dependent datasets into a scalable analytical data platform.

The final Gold datasets will allow business analysts and decision-makers to explore patterns between traffic conditions, PM2.5 concentrations, and weather conditions through an interactive Power BI dashboard.

8. Phase Timeline

Phase 1 — Project Proposal

Deliverables:

- Formal project proposal
- Data-source identification
- Full-load and incremental-load design
- Raw sample files
- High-level Bronze/Silver/Gold design
- Dashboard plan
- GitHub repository

Phase 2 — Pipeline Implementation

Due: 10 October 2026

Expected work:

- API ingestion
- Full-load pipeline
- Incremental-load pipeline
- Bronze implementation
- Silver transformations
- Data-quality validation
- Spark processing
- Spatial and temporal integration

Phase 3 — Analytics & Dashboard

Due: 24 October 2026

Expected work:

- Gold-layer implementation
- Analytical aggregations
- Power BI dashboard
- Performance optimization
- FinOps evaluation
- Final project demonstration